EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor
Course code: MDC401
Year: SE SEM: IV

Experiment No. 7
AIM: Use XSLT (eXtensible Stylesheet Language Transformations) to convert XML data into an HTML table for better presentation.Create an XML file (books.xml) containing book details (Title, Author, Price). Write an XSL file (books.xsl) that transforms the XML data into an HTML table. Link the XSL with XML and open it in a browser to see the formatted output
Name: Ashish Ramesh Patel
Roll Number: 55
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty: Dr. Yogesh Pingle

Aim: Implement user authentication using PHP sessions and cookies and allow users to login and maintain session state across multiple pages.

Theory:

XML (eXtensible Markup Language) is used to store and transport data in a structured and self-descriptive format. However, XML by itself does not define how data should be displayed in a web browser. To present XML data in a readable and user-friendly format, XSLT (eXtensible Stylesheet Language Transformations) is used.

XSLT is a language designed to transform XML documents into other formats such as HTML, XHTML, or plain text. It works by applying transformation rules defined in an XSL stylesheet to an XML document. These rules specify how elements in the XML file should be matched and displayed in the output document.

In this experiment, an XML file named books.xml is created to store book information such as title, author, and price. An XSL file named books.xsl is used to transform this XML data into an HTML table. The XSL file is linked to the XML document so that when the XML file is opened in a browser, the data is displayed in a formatted HTML table.

Algorithm / Procedure:

1. Start the experiment.
2. Create an XML file named books.xml.
3. Define book details such as title, author, and price using XML tags.
4. Create an XSL file named books.xsl.
5. Write XSLT rules to transform XML elements into an HTML table.
6. Link the XSL file to the XML file using the xml-stylesheet instruction.
7. Save both files in the same folder.
8. Open the XML file in a web browser.
9. Observe the formatted HTML table output.
10. Stop the experiment.

Code:

▪ Db.php:

```
<?php

$host = '127.0.0.1';
$user = 'root';
$pass = '';
$dbname = 'mdm\_exp6';

$port = 3306;

$conn = new mysqli($host, $user, $pass, $dbname, $port);
```

```
if ($conn->connect\_error) {  die("Connection failed: " .
        $conn->connect\_error);
}
```

▪ register.php

```
<?php
require 'db.php';
$message = '';

if ($\_SERVER\["REQUEST\_METHOD"] == "POST") {
        $username = trim($\_POST\['username']);
        $email = trim($\_POST\['email']);
        $password = $\_POST\['password'];

        if (empty($username) || empty($email) || empty($password)) {
                $message = "All fields are required!";
        } elseif (!filter\_var($email, FILTER\_VALIDATE\_EMAIL)) {
        $message = "Invalid email format!";
        } else {
                $hashed\_password = password\_hash($password,
                PASSWORD\_DEFAULT);

                $stmt = $conn->prepare("INSERT INTO users (username, email, password)
                VALUES (?, ?, ?)");
                $stmt->bind\_param("sss", $username, $email, $hashed\_password);

                if ($stmt->execute()) {
                        $message = " Registration successful! <a href='login.php'>Login
                        here</a>";
                } else {
                        $message = "Error: " . $stmt->error;
                }

                $stmt->close();
        }
}
$conn->close();
?>

<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>User Registration</title>
   <style>
```

```
        body { font-family: Arial, sans-serif; margin: 50px; }
        .container { max-width: 400px; margin: auto; padding: 20px; border: 1px solid
#ccc; border-radius: 5px; }         input[type="text"], input[type="email"],
input[type="password"] { width: 100%; padding: 10px; margin: 10px 0; box-sizing:
border-box; }         button { background-color: #28a745; color: white; padding: 10px
15px; border: none; cursor: pointer; width: 100%; }
        button:hover { background-color: #218838; }
.message { color: red; margin-bottom: 15px; }
    </style>
</head>
<body>
    <div class="container">
        <h2>User Registration System</h2>
        <?php if($message != '') echo "<div class='message'>$message</div>"; ?>
        <form action="register.php" method="POST">
            <label>Username</label>
            <input type="text" name="username" required>

            <label>Email</label>
            <input type="email" name="email" required>

            <label>Password</label>
            <input type="password" name="password" required>

            <button type="submit">Register</button>
        </form>
    </div>
</body>
</html>
```

▪ login.php

```
<?php
session_start(); // Always start the session at the very top!
require 'db.php'; // Reuse your working database connection

$message = '';

// Check if user is already logged in via Session or Cookie
if (isset($_SESSION['user_id'])) {
    header("Location: dashboard.php");
exit();
} elseif (isset($_COOKIE['remember_user'])) {
    // Auto-login using the cookie
    $_SESSION['user_id'] = $_COOKIE['remember_user'];
$_SESSION['username'] = $_COOKIE['remember_username'];
```

```
    header("Location: dashboard.php");
exit();
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $email = trim($_POST['email']);
    $password = $_POST['password'];
    $remember = isset($_POST['remember']) ? true : false;

    // Fetch user from database
    $stmt = $conn->prepare("SELECT id, username, password FROM users WHERE email
=
?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $result = $stmt->get_result();

    if ($result->num_rows === 1) {
$user = $result->fetch_assoc();

        // Verify the hashed password from Experiment 6
        if (password_verify($password, $user['password'])) {

            // 1. Set Session Variables
            $_SESSION['user_id'] = $user['id'];
            $_SESSION['username'] = $user['username'];

            // 2. Set Cookies if "Remember Me" is checked (expires in 30 days)
if ($remember) {
                setcookie("remember_user", $user['id'], time() + (86400 * 30), "/");
                setcookie("remember_username", $user['username'], time() + (86400 * 30), "/");
            }

            header("Location: dashboard.php"); // Redirect to protected page
exit();
        } else {
            $message = "Incorrect password!";
        }
    } else {
        $message = "No account found with that email!";
    }
    $stmt->close();
}
$conn->close();
?>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Login System</title>
   <style>
      body { font-family: Arial, sans-serif; margin: 50px; }
      .container { max-width: 400px; margin: auto; padding: 20px; border: 1px solid
#ccc; border-radius: 5px; }         input[type="email"], input[type="password"] { width:
100%; padding: 10px; margin:
10px 0; box-sizing: border-box; }
      button { background-color: #007bff; color: white; padding: 10px 15px; border: none;
cursor: pointer; width: 100%; }
      .message { color: red; margin-bottom: 15px; }
   </style>
</head>
<body>
   <div class="container">
      <h2>User Login</h2>
      <?php if($message != '') echo "<div class='message'>$message</div>"; ?>

      <form action="" method="POST">
         <label>Email</label>
         <input type="email" name="email" required>

         <label>Password</label>
         <input type="password" name="password" required>

         <div>
            <input type="checkbox" name="remember" id="remember">
            <label for="remember">Remember Me (Set Cookie)</label>
         </div>
         <br>
         <button type="submit">Login</button>
      </form>
      <p>Don't have an account? <a href="register.php">Register here</a></p>
</div>
</body>
</html>
```

▪ dashboard.php

```
<?php
session_start();

// THE BOUNCER: If the user doesn't have a session VIP wristband, kick them out!
```

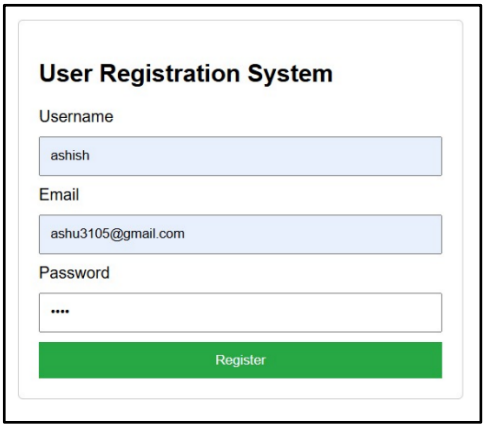
```
if (!isset($_SESSION['user_id'])) {
   header("Location: login.php");
exit();
}
?>

<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>User Dashboard</title>
   <style>
      body { font-family: Arial, sans-serif; margin: 50px; background-color: #f4f4f9;}
.card { background: white; padding: 30px; border-radius: 8px; box-shadow: 0 4px 8px
rgba(0,0,0,0.1); max-width: 600px; margin: auto; text-align: center; }
      .btn-logout { background-color: #dc3545; color: white; padding: 10px 20px;
textdecoration: none; border-radius: 5px; display: inline-block; margin-top: 20px;}
</style>
</head>
<body>
   <div class="card">
      <h2>Welcome to your Dashboard, <?php echo
htmlspecialchars($_SESSION['username']); ?>!</h2>

      <p>You have successfully authenticated using PHP Sessions.</p>
      <p>Try refreshing the page, or opening a new tab. You will stay logged in because
your Session state is being maintained!</p>

      <a href="logout.php" class="btn-logout">Logout</a>
   </div>
</body>
</html>
```

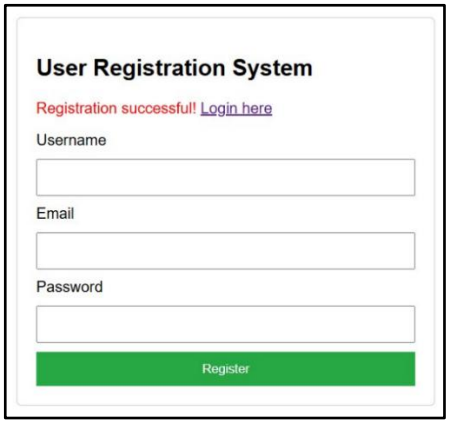
▪ logout.php

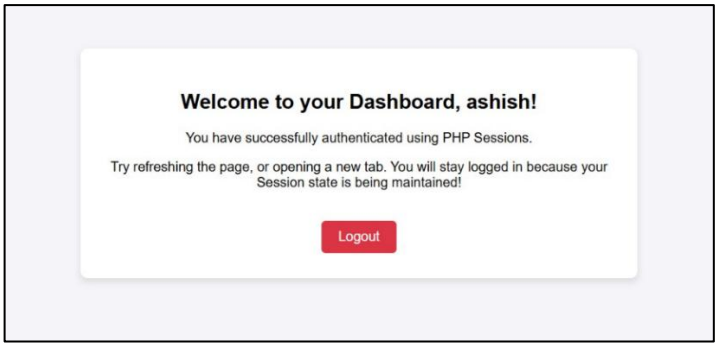
```
<?php
session_start();

// 1. Clear all session variables
$_SESSION = array();

// 2. Destroy the session on the server
session_destroy();

// 3. Delete the "Remember Me" cookies by setting their expiration date to the past (1 hour
ago)
```

```
if (isset($_COOKIE['remember_user'])) {
setcookie("remember_user", "", time() - 3600, "/");
setcookie("remember_username", "", time() - 3600, "/");
}

// 4. Redirect back to login page
header("Location: login.php");
exit();
?>
```

Output:







Conclusion

Thus, XML data was successfully transformed into an HTML table using XSLT. This experiment demonstrates how XSL stylesheets can be used to separate data storage from presentation and improve the readability of XML documents in web browsers.

Questions:

1. **How is XSL linked with XML?**

Ans:
Definition: XSL (Extensible Stylesheet Language) is linked with XML to transform and style XML data.
Key Points:

- XML defines structured data.
- XSL specifies how that data should be displayed (e.g., HTML format).
- Linking is done using the <?xml-stylesheet type="text/xsl" href="style.xsl"?> directive.

2. **What does <xsl:for-each> do?**

Ans:
Definition: <xsl:for-each> is used in XSLT to iterate over a set of XML nodes.
Key Points:

- Applies a template or transformation to each selected node.
- Syntax:

  xml:

  ```
  <xsl:for-each select="book">
    <p><xsl:value-of select="title"/></p>
  </xsl:for-each>
  ```

- Useful for displaying lists (e.g., books, products, records).

3. **Can XSLT convert XML into formats other than HTML?**

Ans:
Definition: Yes, XSLT can transform XML into multiple output formats.
Key Points:

- Commonly used to convert XML → HTML for web display.
- Can also generate **plain text, PDF, or other XML formats**.
- Provides flexibility in presenting structured data in different contexts.